

COMP6441 Week 3 Portfolio

Name: Chuanze zhao

zID:z5659271

Date Submitted: 22/06/2026

1.Setup

Security Everywhere

This week, I noticed that many restaurants now use scanning the QR code to order in order to save the cost of communication between guests and waiters. This method is really convenient. Guests can look at the menu slowly by themselves. They don't need to wait for the waiter all the time, and they won't feel too much pressure when ordering. For restaurants, this can also reduce the workload of waiters and save some of the salary costs of waiters.



But it suddenly occurred to me that there are certain security risks in this way. The QR code itself is unreadable to people. What we see is just a pattern, and we can't directly know which website it is connected to. Therefore, in most cases, what users believe in is not the QR code itself, but its location. For example, if it is posted on the restaurant table, users will naturally think that it is a QR code used to order food.

If the attacker replaces the real QR code with a fake QR code, the user may be guided to a phishing website. Although most restaurants only order online and pay at the front desk, some restaurants will require customers to pay online first. In this case, if the fake ordering website requires users to enter the bank card number and CV, it may cause the bank card to be stolen.

QR code replacement is a case of big trouble caused by small details in life, because the ordering website of the restaurant does not have obvious Url characteristics like government websites or bank websites, and users cannot analyse whether the website is a phishing page through the domain name. Even if most restaurants use the same ordering system to order, it is difficult for customers who dine for the first time to judge that this web page is safe at a glance. Moreover, when users order in the restaurant, they are usually not in a state of strong security vigilance. They just want to complete the order as soon as possible, so it is easy to ignore the details of the URL, merchant name or payment page.

I think the improvement method is that the restaurant can regularly check whether the QR code on the desktop has been covered or replaced, or the user can check the thickness of the QR code on the desktop before scanning the QR code. If there are two layers, don't scan the QR code anymore. Users should also confirm whether the web page is reliable before entering the bank card information, and try to use third-party payment platforms such as Apple Pay or PayPal, which is much more information secure than entering the bank card directly.

2. Humans, Measuring

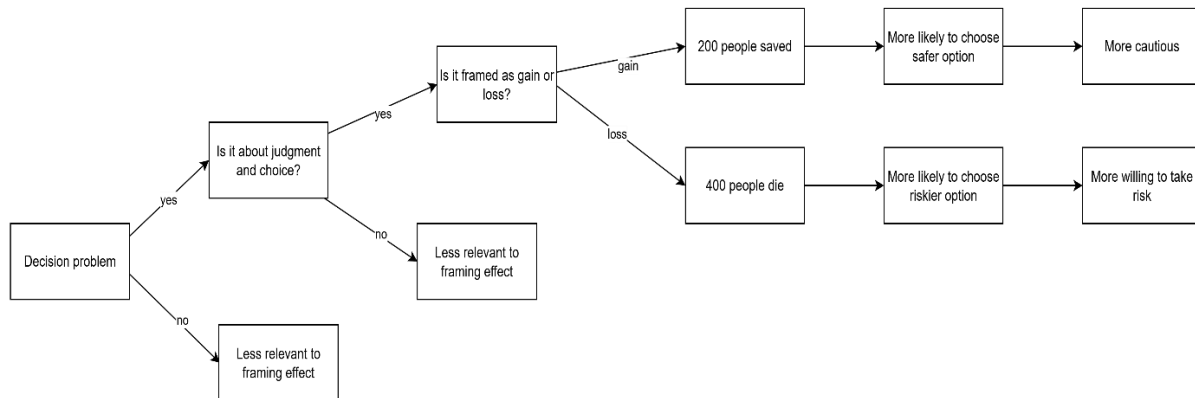
Research a Cognitive Vulnerability

The thesis I chose is ***The Framing of Decisions and the Psychology of Choice by Tversky and Kahneman***. This paper studies the framing effect, that is, the framing

effect. Simply put, if the same decision-making problem is described in different ways, it may affect people's judgement and choice. The paper points out that human choices are not always completely rational, and the judgement process will be influenced by the framework of language expression, emotions, gains and losses.

The most classic example in the paper is the Asian disease problem. The topic assumes that a disease can cause 600 deaths. When the option was described as "certain to save 200 people", most people chose a more conservative plan. But when the same result was described as "400 people will die", more people chose the adventure plan. Although the results of these two versions are very similar, because one emphasizes gain and the other emphasizes loss, people's choices have changed significantly. This shows that people tend to be more conservative when emphasizing benefits; while people may become more radical and riskier when emphasizing losses.

Simple diagram helps explain main ideas:



This cognitive bias is highly related to network security. Attackers can use negative frameworks and urgent language in scam messages. Classic statements are such as "a stranger has tried to log in your account recently" or "Your account has been frozen and needs to click the link to unfreeze it". These languages focus on making users believe that they are gradually losing control of the account and making irrational actions. Being overly nervous and afraid of losing is likely to ignore any risks that may occur by clicking on the link.

Attackers do not necessarily need high-level technology. They can put pressure on users just by changing the way the information is expressed to achieve the results they expect.

Therefore, the framing effect is not just a psychological cognitive bias, but also a kind of cognitive vulnerability. People simply don't process information the way a machine would. Our judgements are shaped by context, emotion, and how the question is framed in the first place. Therefore, the focus of security design should not be on whether the user himself is rational enough and whether the user's subjective judgement is safe. The Defense method that security engineers should focus on is to

reduce the choice of irrational judgement in users' extreme situations. For example, the system can add additional warnings to suspicious links to give more obvious prompts, add a confirmation page before jumping to high-risk phishing website links, and remind users to pay special attention to link security. This is also a function of most browsers at present. Through this function, users can effectively avoid making choices that are harmful to their own safety.

Enigma Machine

This week I made a paper Enigma machine and used it to encrypt and decrypt some messages. Through this process, I can more intuitively understand that Enigma is not a simple substitution cipher, but a mechanical encryption system that changes with each input.

I first watched the YouTube video and learned the basic working principle of Enigma: Enigma will convert the input letter into another letter through multiple rotors, and the position of the rotor will change after each letter is entered. Therefore, when the same letter is entered in different positions, different output results may be obtained. This is more difficult to crack than ordinary substitution cipher, because its replacement rules are not fixed, but will change with the movement of the rotor.

After understanding the basic principle, I bought a Doritos potato chip bucket as the barrel of Enigma, and printed a paper password template and pasted it on it. After the assembly is completed, I use the example provided at the bottom of the web page to test it.

The test information is: Favorite Word: XBEBNxBVUP, Rotor Order: II, III, I, Reflector: B, Starting Position: JJJ. After decrypting with the same settings, I got the answer PANOPTICON, which shows that my paper Enigma machine can basically be used normally.



Then, I went to the comment section to find the passwords of two students to crack. The information of the first student is: Favorite Word: TZFNIUMEQW, Reason: Is a very popular sport, Rotor Order: II, III, I, Reflector: B, Starting Position: AUV. After cracking, the answer I got was basketball.

The information of the second classmate is: Favorite Word: JTKUPR, Reason: Coffee is my favorite word because I drink it almost every Day. It helps me stay focused when studying and has become part of my daily routine. Rotor Order: I, II, III, Reflector: B, Starting Position: AAA. After cracking, the answer I got was coffee.

I also left a puzzle for myself: Favorite Word: IVUDLN, Reason: My favorite drink, Rotor Order: I, II, III, Reflector: B, Starting Position: QLR. The answer to this puzzle should be icetea.

After solving and creating puzzles, I found that the security of Enigma is relatively high. Compared with the shift cipher of Caesar cipher tried last week, it uses rotor order, starting position, reflector, and rotor movement after each input. It made me realize that even a small mechanical change, like rotating the rotor after each letter, can make the cipher much harder to break.

Estimate Attacker Power

Attacker	Budget	Hardware	Bits of work per 24 hours	Time to do 128 bits of work
Office Worker	\$200 (plus company provided laptop)	Quadcore i5 laptop	51	2 ⁷⁷ days 2 ⁶⁸ years
Solo teenage Hacker	\$10k	solid-tier gaming pc	2 ⁴⁵ *2 ¹⁷ = 2 ⁶²	2 ⁶⁶ days 2 ⁵⁷ years
Small Organisation	\$50k	Around 5 solid-tier gaming PCs / GPU machines	2 ⁶⁴	2 ⁵⁵ years
Large Organisation	\$1M	100 solid-tier gaming PCs / GPU machines	2 ⁶⁹	2 ⁵⁰ years
Large botnet	Not many cost but able to access many pc	Maybe 100000 mid-tier pc	2 ⁶⁸	2 ⁵¹ years
Professional bitcoin miner	10M dollars	1000 solid-tier gaming PCs / GPU machines	2 ⁷²	2 ⁴⁷ years

Foreign Intelligence Agency	100M government funds	10000 H100 Gpu	2^{79}	2^{40} years
-----------------------------	-----------------------	----------------	----------	----------------

First, I'll use the "Office Worker" calculation example provided on the webpage as the basis for this analysis. The webpage mentions that a 2.5GHz quad-core laptop can perform 2^{34} operations per second. There are approximately 2^{17} seconds in a 24-hour day, so the daily workload is 2^{51} operations. Therefore, brute-forcing a problem of the 2^{128} order would take 2^{77} days, or approximately 2^{68} years.

Next is the "Solo Teenage Hacker." I'll assume their device's cracking power is 2^{11} times that of an average laptop, which is about 2^{45} operations per second, resulting in a daily workload of 2^{62} . Therefore, completing 2^{128} would take 2^{66} days, or approximately 2^{57} years.

For a Small Organization, I assume it has a budget of \$50k, which can purchase about 5 similar high-performance machines. The total daily workload is approximately 2^{64} , so completing 2^{128} would take about 2^{55} years.

A Large Organization has a budget of \$1M. Estimating at \$10k per machine, it would have roughly 100 machines, with a daily workload of approximately 2^{69} , requiring about 2^{50} years to complete.

A large botnet is a special case; its budget is actually very difficult to estimate directly because it does not need to purchase hardware itself. Attackers can secretly install backdoor programs on ordinary users' computers by spreading malware, bundling it with pirated software, or using phishing links, and then harness the computing power of these machines without the users' knowledge.

Therefore, even if the attacker's own budget is modest, the actual computing power they can mobilize may be enormous. I'll assume a large botnet can control approximately 100,000 ordinary users' computers. Estimating 2^{51} per machine per day, the total workload is approximately 2^{68} , and it would take about 2^{51} years to complete 2^{128} .

A professional Bitcoin miner with approximately \$10M in funding could own about 1,000 high-performance machines, with a daily hash rate of about 2^{72} , requiring approximately 2^{47} years.

A foreign intelligence agency has \$100M in government funding and can mobilize about 10,000 H100 GPUs. Since the parallel computing power of an H100 is approximately 2^4 times that of a standard gaming GPU, its daily workload is about 2^{79} , and it would take approximately 2^{40} years to complete 2^{128} .

Based on these estimates, even if an attacker continuously increases their budget and piles on hardware, brute-forcing 2^{128} is essentially impossible. The fundamental reason is this: the increase in computing power resulting from a larger budget is linear, purchasing 10 times as much equipment will, at most, speed up the process by a factor of 10. However, the difficulty of cracking a 128-bit key is exponential, meaning the two growth curves are fundamentally different. Therefore, most attackers do not attempt brute-force attacks; instead, they specifically seek out system vulnerabilities, misconfigurations, or weak passwords to carry out targeted attacks, which is a more practical approach and offers a higher chance of success.

Translated with DeepL.com (free version)

3. Analysis and Reflection

Tutorial 3: Houdini

This week's tutorial case study was about Houdini and how he tried to test whether mediums could really communicate with the dead. The basic problem was that Houdini did not want fake mediums to claim that they had received messages from his spirit after his death.

If a medium could just guess vague answers and use Bess's reactions as hints, then the result would not really prove anything. The main question for us was how Houdini and Bess could design a protocol to check whether a message really came from Houdini, rather than being invented by a fake medium.

I think one reason people believe in mediums is that grief can make people less rational. When someone loses a loved one, they may want to believe comforting messages more than they normally would. This also links to the cognitive bias idea we discussed before. People under sadness, fear, or pressure may not judge information as carefully. A medium could say something vague but emotionally meaningful, and the listener might fill in the missing details by themselves. If the message sounds close to what they want to hear, they may believe it is real.

In our discussion, one important idea was to hide or obfuscate the final solution. The secret should not be something too obvious or easy to guess. For example, if Houdini and Bess chose a very common phrase, a fake medium might guess it by using public

information about Houdini, common emotional phrases, or Bess's reactions. This reminds me of attackers trying common passwords first, such as birthdays, 000000, or password. The problem is not only whether the secret exists, but whether the secret has enough uncertainty to resist guessing.

Another part of the protocol should be extra verification. If the medium happens to guess one word correctly, that should not be enough to pass the whole test. The protocol needs more than one layer, because a single secret word is too fragile. This is similar to modern login systems: knowing the password may not be enough if the system also asks for an email code or phone verification. In Houdini's case, the extra layers could make it much harder for a fake medium to succeed by luck.

For the time limit, my understanding is that it is not mainly about making the medium nervous. It is about stopping the medium from trying many answers and watching Bess's face for hints. If Bess reacts differently when the medium says something close to the truth, the medium may slowly learn the secret. A time limit or attempt limit reduces this kind of trial-and-error attack. This is like rate limiting or temporary lockout in modern systems, where too many failed attempts can trigger a lock to prevent brute-force attacks.

For Q2, our protocol idea was that the answer should be verified as a complete sequence. The medium should not be allowed to test the answer word by word. Even if some words are correct, the order should also matter. A better design is that the medium gives the whole sequence at once, and Bess only says whether the complete answer is correct or incorrect. She should not say which part is right or wrong, because that feedback would help the medium build the answer slowly.

Overall, the main point is that Bess should not give the medium useful feedback, and the medium should only get a limited chance to provide the complete secret sequence.

4.Wrap up

Submission Instructions

To submit a valid portfolio entry, complete the activities, save your work as a PDF, and upload it to the relevant week's submission page on Moodle.